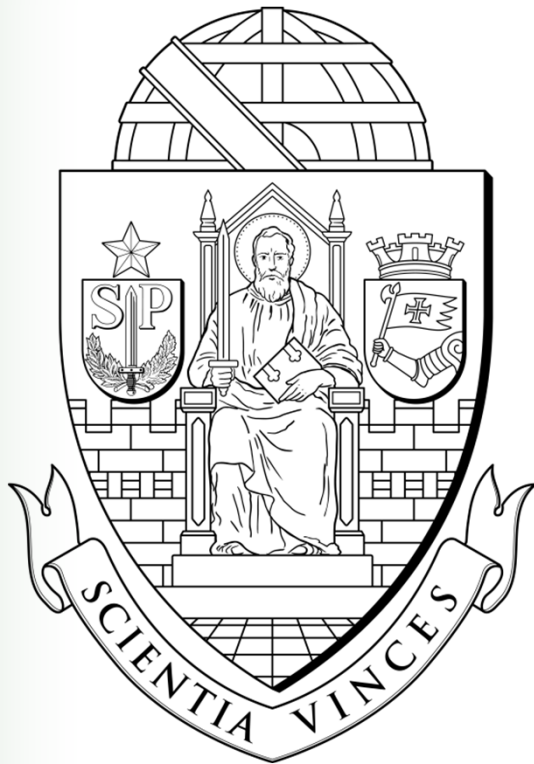


Organização de Arquivos



USP

- Nesta aula são introduzidos conceitos sobre organização de arquivos, intercalação e o modelo cossequencial de processamento de arquivos

Sistema de Arquivos

- ❑ O Sistema de Arquivos (*file system*) é um software responsável por organizar setores do disco em arquivos e diretórios
- ❑ O Sistema de Arquivos gerencia quais setores pertencem a um determinado arquivo bem como quais setores não estão sendo utilizados
- ❑ Assim, o Sistema de Arquivos permite trabalhar com arquivos de forma lógica, sem se preocupar com os detalhes do armazenamento físico

Arquivos

- ❑ Em nosso contexto, um arquivo é definido como um conjunto de **registros**
- ❑ Os registros são formados por unidades de informação denominadas **campos**
- ❑ Em geral, há um campo especial denominado **chave**, que identifica univocamente cada registro
- ❑ Cada arquivo possui associado um **ponteiro de arquivo** que indica a posição (registro) do arquivo onde será efetuada a próxima leitura/gravação
- ❑ Os registros são numerados sequencialmente a partir da unidade (1, 2, 3, ...)
 - Portanto, um arquivo vazio possui zero registros

Arquivos

□ Por exemplo, os registros de um arquivo de funcionários poderiam incluir os seguintes campos:

- Número de matrícula (chave)
- Nome
- Cargo
- Grau de escolaridade
- Sexo (M,F)
- Local
- Estado civil (S, C)
- Salário

Arquivos

❑ No Exemplo

- Cada linha é um registro (5 registros)
- Cada coluna é um campo (8 campos)
- O ponteiro de arquivo (►) encontra-se no 3o. registro

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
► 3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Especificação

- ❑ Operações elementares em arquivos
 - abrir (*open*)
 - fechar (*close*)
 - ler (*read, input*)
 - escrever (*write, output*)
 - testar pelo final-de-arquivo (*eof = end-of-file*)
 - posicionar (*seek*) o arquivo em um determinado registro
 - encontrar a posição atual do ponteiro de arquivo (registro atual ou corrente)

Criação

File::File(string filename, Mode modo)

- ❑ *pré-condição*: o arquivo não esteja aberto
- ❑ *pós-condição*: o ADT é associado ao arquivo em disco de nome *filename* e o arquivo é aberto no modo especificado
 - enum Mode {read, write, readwrite};
- ❑ Em geral, arquivos abertos nos modos **read** ou **readwrite** devem existir em disco; no modo **write** todo conteúdo do arquivo é descartado (se existir) e o arquivo é aberto completamente sem registros (vazio)

Finalização

File::~~File();

- *pré-condição:* O arquivo tenha sido criado
- *pós-condição:* O arquivo é fechado, garantindo que toda informação seja efetivamente escrita em disco; após fechado, o arquivo correspondente do sistema operacional permanece residente em disco

Status

`bool File::eof();`

❑ *pré-condição:* Arquivo tenha sido criado

❑ *pós-condição:* função retorna **true** se a próxima leitura/gravação será efetuada após o final do arquivo, ou seja, se o ponteiro de arquivo encontra-se no final do mesmo; **false** caso contrário

Operações Básicas

`void File::write(FileEntry x);`

- ❑ *pré-condição:* Arquivo já tenha sido criado nos modos **write** ou **readwrite**
- ❑ *pós-condição:* O item **x** é armazenado na posição (registro) onde se encontra o ponteiro de arquivo, sobrescrevendo qualquer informação anterior; o ponteiro de arquivo avança para o próximo registro

Operações Básicas

`void File::write(FileEntry x);`

- ❑ *pré-condição:* Arquivo já tenha sido criado nos modos **write** ou **readwrite**
- ❑ *pós-condição:* O item **x** é armazenado na posição (registro) onde se encontra o ponteiro de arquivo, substituindo qualquer informação e o ponteiro de arquivo avança para o próximo registro

O tipo **FileEntry** depende da aplicação e pode variar desde um simples caracter ou número até uma **struct** ou **class** com muitos campos

Operações Básicas

`void File::read(FileEntry &x);`

- ❑ *pré-condição:* Arquivo já tenha sido criado nos modos **read** ou **readwrite**
- ❑ *pós-condição:* O registro onde se encontra o ponteiro de arquivo é lido e copiado para a variável **x**; o ponteiro de arquivo avança para o próximo registro

Operações Básicas

`void File::seek(int n);`

- *pré-condição:* Arquivo já tenha sido criado
- *pós-condição:* O ponteiro de arquivo é posicionado no registro de número **n**, sendo que qualquer operação de leitura/escrita subsequente será efetuada no registro **n**

Outras Operações

`int File::size();`

- ❑ *pré-condição:* Arquivo já tenha sido criado
- ❑ *pós-condição:* a função retorna o número de registros no arquivo

Outras Operações

`int File::pos();`

- ❑ *pré-condição:* Arquivo já tenha sido criado
- ❑ *pós-condição:* a função retorna o registro atual (corrente) em que o arquivo se encontra

Pontos Importantes

- ❑ Um arquivo pode aberto em três modos elementares:
 - Apenas para leitura (*read* ou *read only*)
 - Apenas para escrita (*write* ou *write only*)
 - Para leitura e escrita (*read-write*)
- ❑ O acesso aos registros em um arquivo pode ocorrer de forma
 - Sequencial (um registro é lido/escrito após o anterior)
 - Aleatória (posiciona-se em um registro para posterior leitura/gravação) ou
 - uma combinação de ambas
- ❑ Como regra geral nas linguagens de programação, todo arquivo antes de ser utilizado deve ser **aberto**; ao término das operações com o arquivo ele deve ser **fechado**
- ❑ Em C++ o fechamento de arquivo é opcional, já que arquivos são objetos em C++ e seu *finalizador* se encarrega de fechar o arquivo quando o objeto deixa de existir; entretanto é uma boa prática de programação fechar um arquivo quando ele não é mais necessário para economizar recursos do sistema operacional

Organização de Arquivos

- ❑ O objetivo da organização de arquivos é proporcionar meios para a recuperação e atualização de registros
- ❑ A atualização de um registro pode incluir sua remoção, alteração de alguns dos seus campos ou a inserção de um registro completamente novo
- ❑ A recuperação de registros ocorre por meio de **consultas**

Tipos de Consultas

- ❑ Existem quatro tipos de consultas em arquivos:
 - Q1: Consulta simples: especifica-se o valor de um único campo
 - Q2: Consulta em intervalo: especifica-se um intervalo de valores para um único campo
 - Q3: Consulta funcional: especificam-se alguns valores determinados dentro do arquivo (exemplo: soma, média, desvio-padrão)
 - Q4: Consulta booleana: uma combinação booleana de Q1-Q3 utilizando os operadores lógicos **and**, **or** e **not**

Tipos de Consultas

- ❑ No exemplo, recupere os registros de todos os funcionários com:
- Q1: Sexo = 'M'
 - Q2: Salário > 9000
 - Q3: Salário > média dos salários de todos os funcionários
 - Q4: (Sexo = 'F' **and** Cargo = 'Programador') **or** (Matricula > 700 **and** Sexo = 'M')

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Tipos de Consultas

❑ No exemplo, recupere os registros de todos os funcionários com:

- **Q1: Sexo = 'M'**
- Q2: Salário > 9000
- Q3: Salário > média dos salários de todos os funcionários
- Q4: (Sexo = 'F' **and** Cargo = 'Programador') **or** (Matricula > 700 **and** Sexo = 'M')

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Tipos de Consultas

- ❑ No exemplo, recupere os registros de todos os funcionários com:
- Q1: Sexo = 'M'
 - **Q2: Salário > 9000**
 - Q3: Salário > média dos salários de todos os funcionários
 - Q4: (Sexo = 'F' **and** Cargo = 'Programador') **or** (Matricula > 700 **and** Sexo = 'M')

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Tipos de Consultas

- ❑ No exemplo, recupere os registros de todos os funcionários com:
- Q1: Sexo = 'M'
 - Q2: Salário > 9000
 - **Q3: Salário > média dos salários de todos os funcionários**
 - Q4: (Sexo = 'F' and Cargo = 'Programador') or (Matricula > 700 and Sexo = 'M')

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Tipos de Consultas

- ❑ No exemplo, recupere os registros de todos os funcionários com:
- Q1: Sexo = 'M'
 - Q2: Salário > 9000
 - Q3: Salário > média dos salários de todos os funcionários
 - **Q4: (Sexo = 'F' and Cargo = 'Programador') or (Matricula > 700 and Sexo = 'M')**

Registro	E#	Nome	Cargo	Esc.	Sexo	Local	E.C.	Salário
1	800	Austin	programador	2	F	Campinas	S	10.000
2	510	William	analista	3	M	Campinhas	C	15.000
3	950	Melissa	analista	3	F	Vinhedo	S	12.000
4	750	Hawkins	programador	2	M	Campinas	S	12.000
5	620	Newton	programador	2	M	Vinhedo	C	9.000

Organização de Arquivos

- ❑ É claro que, para cada tipo de consulta, se o arquivo não estiver organizado adequadamente, é necessário uma ou mais varreduras completas (do início ao final) do arquivo
- ❑ Uma primeira ideia seria organizar um arquivo ordenando-o segundo algum campo
- ❑ Um método comum de ordenação consiste na ordenação (ou classificação) por intercalação
- ❑ A intercalação pressupõe dois arquivos ordenados por algum campo, juntando-os em um único arquivo final, também ordenado
- ❑ Em termos gerais a intercalação (*merge*) é o processo de formar uma lista de saída contendo **todos** os itens de duas (ou mais) listas de entrada

Algoritmo de Intercalação

`procedure merge (X, Y, Z)`

- ❑ *pré-condição*: assume dois arquivos de entrada X e Y ordenados pelo campo chave
- ❑ *pós-condição*: gera como saída o arquivo Z também ordenado por chave composto pelos registros de X e Y
- ❑ O procedimento precisa de 3 variáveis do tipo arquivo: duas variáveis de entrada para os arquivos X e Y e uma variável de saída para o arquivo Z
- ❑ No algoritmo, assume-se que os registros possuem um campo chave denominado **key**

Algoritmo de Intercalação

X

	key	nome	sexo
▶	1	walter	m
	5	gustavo	m
	7	ronaldo	m

Y

	key	nome	sexo
▶	3	jaqueline	f
	4	cláudia	f
	10	ana	f
	12	cristina	f

Z

	key	nome	sexo
▶			

Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m

Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f

Z

key	nome	sexo
1	walter	m

Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m

Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f

Z

key	nome	sexo
1	walter	m
3	jaqueline	f

Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m

Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f

Z

key	nome	sexo
1	walter	m
3	jaqueline	f
4	cláudia	f

Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m

Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f

Z

key	nome	sexo
1	walter	m
3	jaqueline	f
4	cláudia	f
5	gustavo	m

Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m



Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f



Z

key	nome	sexo
1	walter	m
3	jaqueline	f
4	cláudia	f
5	gustavo	m
7	ronaldo	m



Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m



Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f



Z

key	nome	sexo
1	walter	m
3	jaqueline	f
4	cláudia	f
5	gustavo	m
7	ronaldo	m
10	ana	f



Algoritmo de Intercalação

X

key	nome	sexo
1	walter	m
5	gustavo	m
7	ronaldo	m



Y

key	nome	sexo
3	jaqueline	f
4	cláudia	f
10	ana	f
12	cristina	f

Z

key	nome	sexo
1	walter	m
3	jaqueline	f
4	cláudia	f
5	gustavo	m
7	ronaldo	m
10	ana	f
12	cristina	F



Algoritmo de Intercalação

```
procedure merge(X,Y,Z)
```

```
  Leia registro x de X
```

```
  Leia registro y de Y
```

```
  while not X.eof() and  
    not Y.eof() do
```

```
    if x.key < y.key then
```

```
      Escreva x em Z
```

```
      Leia x do arquivo X
```

```
    else
```

```
      Escreva y em Z
```

```
      Leia y do arquivo Y
```

```
    endif
```

```
  endwhile
```

```
  while not X.eof() do
```

```
    Escreva x em Z
```

```
    Leia x do arquivo X
```

```
  endwhile
```

```
  while not Y.eof() do
```

```
    Escreva y em Z
```

```
    Leia y do arquivo Y
```

```
  endwhile
```

```
end merge
```

Algoritmo de Intercalação

```
procedure merge(X, Y, Z)

  X.read(x);
  Y.read(y);
  while not X.eof() and
        not Y.eof() do
    if x.key < y.key then
      Z.write(x);
      X.read(x);
    else
      Z.write(y);
      Y.read(y);
    endif
  endwhile
```

```
    while not X.eof() do
      Z.write(x);
      X.read(x);
    endwhile

    while not Y.eof() do
      Z.write(y);
      Y.read(y);
    endwhile

  end merge
```

Algoritmo de Intercalação (Usando Modelo Cossequencial)

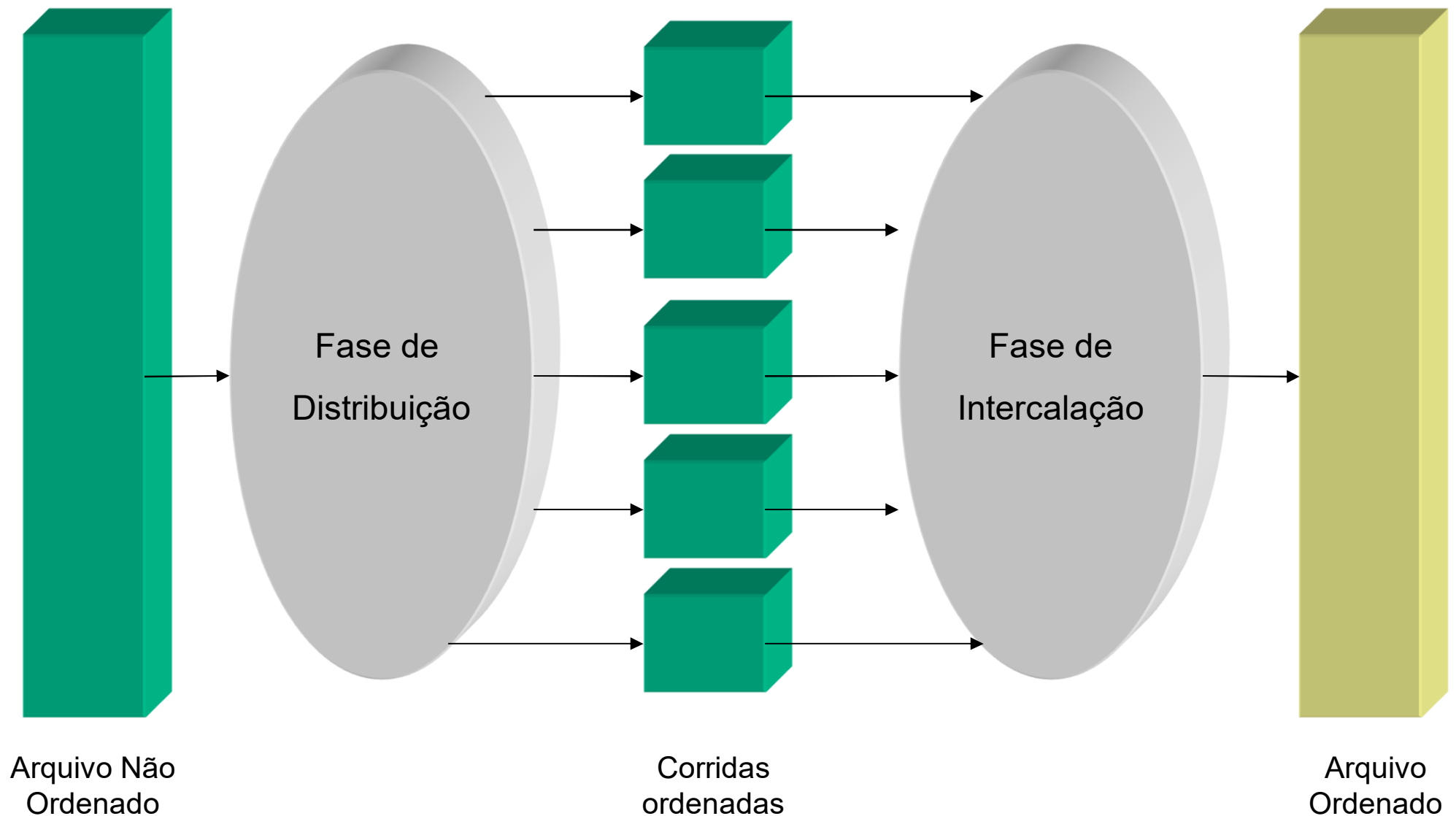
```
procedure merge (X, Y, Z)

  X.read(x);
  Y.read(y);
  acabou ← X.eof() and e.Eof();
  while not acabou do
    if x.key < y.key then
      Z.write(x);
      X.read(x);
    else
      Z.write(y);
      Y.read(y);
    endif
    acabou ← X.eof() and Y.eof();
  endwhile
```

Classificação por Intercalação

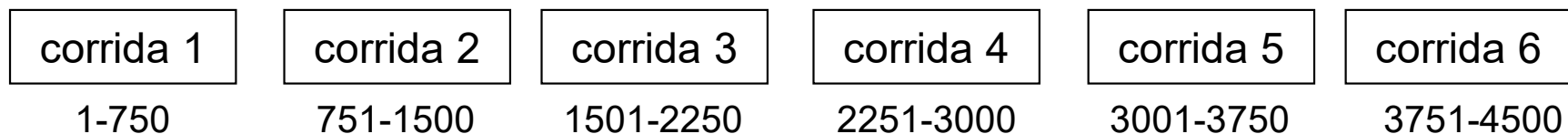
- ❑ A Classificação por Intercalação é o método mais comum de ordenação nos dispositivos de memória secundária
- ❑ Esse método consiste essencialmente de duas fases distintas
 - Na primeira (distribuição) os segmentos do arquivo de entrada são classificados usando um bom método de ordenação em memória principal
 - ❖ Esses segmentos classificados, conhecidos como corridas (*runs*) estão sendo gravados na memória secundária na medida em que são gerados
 - Na segunda (intercalação) as corridas geradas na fase anterior são intercaladas, até esgotar as corridas

Classificação por Intercalação

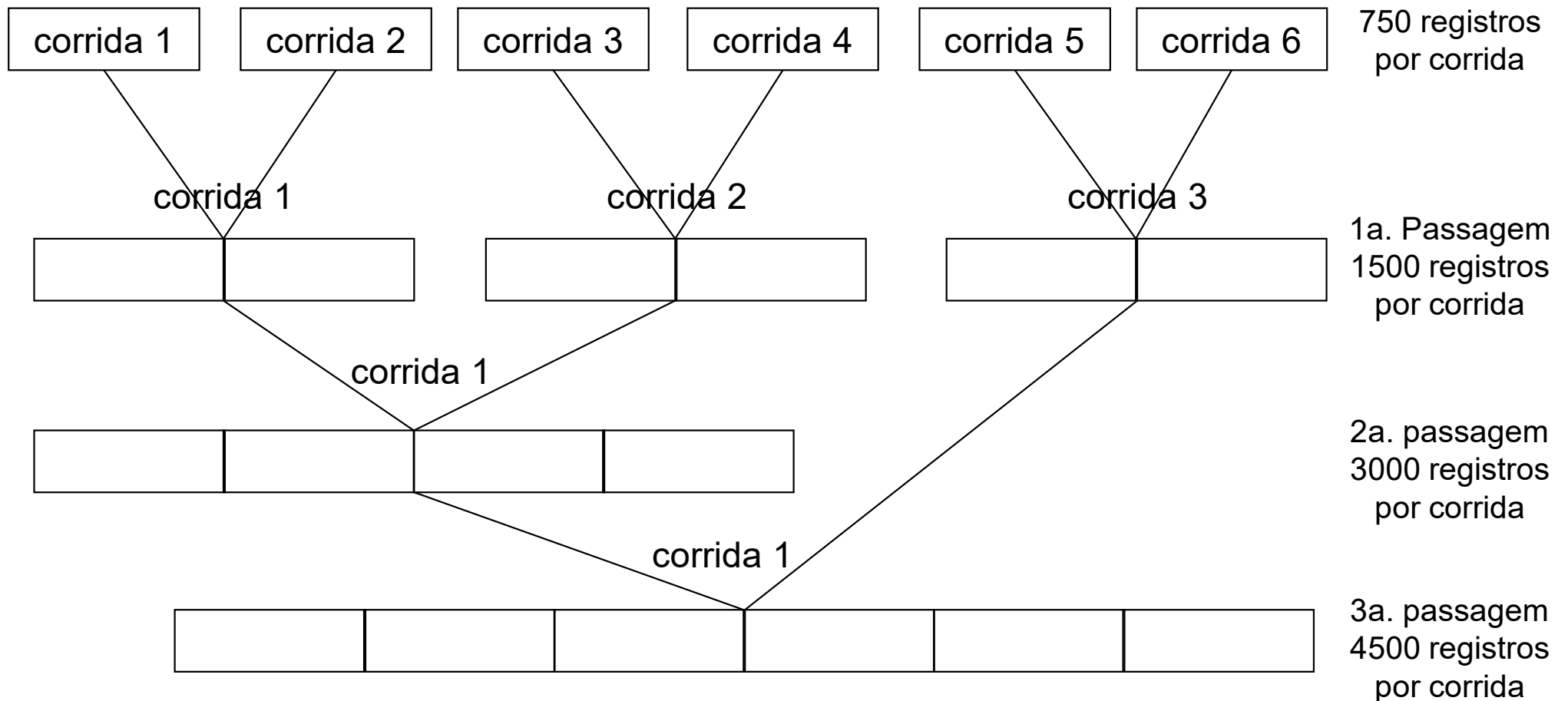


Classificação por Intercalação

- ❑ Por exemplo, deve ser classificado um arquivo contendo 4.500 registros $A_1, \dots, A_{4500}$ em um computador com a memória interna com capacidade de classificar no máximo 750 registros
- ❑ Ordenando de cada vez 750 registros, são necessárias seis corridas $R_1, R_2, \dots, R_6$



Classificação por Intercalação



Exemplo

- ❑ Considere um arquivo com as seguintes chaves

F	45	56	12	43	95	19	8	67
---	----	----	----	----	----	----	---	----

- ❑ que é particionado em duas corridas

R_1	45	56	12	43
-------	----	----	----	----

R_2	95	19	8	67
-------	----	----	---	----

- ❑ Que são ordenadas em memória principal

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'							
------	--	--	--	--	--	--	--

Exemplo

□ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

□ são intercaladas...

F'	8						
------	---	--	--	--	--	--	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'	8	12						
------	---	----	--	--	--	--	--	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'	8	12	19					
------	---	----	----	--	--	--	--	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'	8	12	19	43				
------	---	----	----	----	--	--	--	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'	8	12	19	43	45			
------	---	----	----	----	----	--	--	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas...

F'	8	12	19	43	45	56		
------	---	----	----	----	----	----	--	--

Exemplo

□ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

□ são intercaladas...

F'	8	12	19	43	45	56	67	
------	---	----	----	----	----	----	----	--

Exemplo

❑ As corridas ordenadas

R_1	12	43	45	56
-------	----	----	----	----

R_2	8	19	67	95
-------	---	----	----	----

❑ são intercaladas formando uma nova corrida

F'	8	12	19	43	45	56	67	95
------	---	----	----	----	----	----	----	----

❑ Neste caso, essa corrida é a corrida final, resultando no arquivo F ordenado (F')

Modelo Cossequencial

- ❑ A intercalação é um exemplo de uma operação **cossequencial** de processamento de arquivos
- ❑ As operações cossequenciais envolvem o processamento coordenado de duas ou mais entradas sequenciais, de modo a produzir uma única saída, operação muito comum em arquivos
- ❑ Exemplos de operações cossequenciais são a união (*merge*) e a intersecção (*match*) de duas ou mais entradas
 - Por exemplo, a operação de união é a base do processo de ordenação de arquivos muito grandes, cujo índice não cabe em memória principal

Algoritmo Cossequencial de Intercalação (Merge)

- ❑ Iniciar (abrir arquivos de entrada e de saída)
- ❑ Leia o primeiro item de cada lista
- ❑ if *todas as listas atingiram fim-de-arquivo* then acabou←true else acabou←false endif
- ❑ while not acabou do
 - Compare os itens atuais de cada uma das listas
 - if os itens são iguais then
 - ❖ Processe o item
 - ❖ Leia o próximo item de cada lista
 - else
 - if o item da lista A é menor do que o item da lista B then
 - ❖ Processe o item da lista A
 - ❖ Leia o próximo item da lista A
 - else
 - if o item da lista A é maior do que o item da lista B then
 - ❖ Processe o item da lista B
 - ❖ Leia o próximo item da lista B
 - endif
 - if *todas as listas atingiram fim-de-arquivo* then acabou ← true endif
- ❑ endwhile
- ❑ Finalizar (fechar arquivos)

Algoritmo Cossequencial de Intersecção (Match)

- ❑ Iniciar (abrir arquivos de entrada e de saída)
- ❑ Leia o primeiro item de cada lista
- ❑ if *uma das listas atingiu fim-de-arquivo* then acabou←true else acabou←false
endif
- ❑ while not acabou do
 - Compare os itens atuais de cada uma das listas
 - if os itens são iguais then
 - ❖ Processe o item
 - ❖ Leia o próximo item de cada lista
 - ❖ if *uma das listas atingiu fim-de-arquivo* then acabou ← true endif
 - else
 - if o item da lista A é menor do que o item da lista B then
 - ❖ Leia o próximo item da lista A
 - ❖ if *Lista A atingiu fim-de-arquivo* then acabou ← true endif
 - else
 - if o item da lista A é maior do que o item da lista B then
 - ❖ Leia o próximo item da lista B
 - ❖ if *Lista B atingiu fim-de-arquivo* then acabou ← true endif
 - endif
- ❑ endwhile
- ❑ Finalizar (fechar arquivos)

Modelo Cossequencial

- ❑ Dois ou mais arquivos de entrada devem ser processados simultaneamente para produzir um (ou mais) arquivo de saída
- ❑ Cada arquivo é ordenado sobre um ou mais campos chave, e todos os arquivos são ordenados do mesmo modo sobre esses campos
- ❑ Para ajudar no gerenciamento condições especiais de início e final de arquivo é interessante, embora não obrigatório, estabelecer valores altos (*high-values*, $+\infty$) e baixos (*low-values*, $-\infty$) que são maiores e menores do que qualquer chave, respectivamente
- ❑ Registros são processados de acordo com a ordem lógica de ordenação; a ordem física não é relevante ao modelo, mas na prática pode ser importante para a implementação pois a ordenação física pode ter grande impacto na eficiência do processamento
- ❑ Para cada arquivo existe um único registro corrente, cuja chave é aquela disponível no *loop* de sincronização principal; o modelo não proíbe que se olhe para a frente ou para trás no arquivo, desde que feito fora do *loop* e sem afetar a estrutura de sincronização
- ❑ Registros são manipulados somente em memória principal já que não se pode alterar um registro diretamente no arquivo em disco

Intercalação em Múltiplas Vias

(Multiway Merging)

- Uma variação comum do modelo para processos cossequenciais é a **intercalação em K-vias** (*K-way Merge*) na qual **K** arquivos de entrada ordenados são intercalados, gerando um único arquivo de saída
- O valor **K** é a ordem da intercalação

Algoritmo Cossequencial de Intercalação K-vias

- ❑ Iniciar (abrir arquivos de entrada e de saída)
- ❑ Leia o primeiro item de cada lista
- ❑ if *todas as listas atingiram fim-de-arquivo* then
 acabou ← true else acabou ← false endif
- ❑ while not acabou do
 - minItem ← encontre menor item dentre as K listas
 - Processe minItem
 - for i ← 1 to K do (em incrementos de +1)
 - ❖ if *item da lista i é igual a minItem* then
 - Leia o próximo item da lista i
 - ❖ endif
 - next i
 - if *todas as listas atingiram fim-de-arquivo* then acabou ← true
 endif
- ❑ endwhile
- ❑ Finalizar (fechar arquivos)

Intercalação K-vias

□ Para encontrar o menor item

- Busca linear: encontre o menor dos K itens procurando sequencialmente desde 1 até o K-ésimo item
- Busca usando *heap*: K itens (cada um de uma das listas) são mantidos em um *heap* (sob a disciplina: raiz = menor valor); mova o menor item para minItem, substituindo o item movido pelo seu vizinho (próximo item na lista da qual o item movido veio) e então refaça o *heap*

Resumo

- ❑ Discos correspondem à dispositivos de armazenamento secundário
- ❑ O tempo de acesso a disco é muito grande quando comparado ao tempo de acesso a registradores na CPU
- ❑ Para alterar um registro em disco é necessário primeiro que ele seja trazido para memória principal; o registro é alterado em memória principal e novamente escrito em disco
- ❑ O modelo cossequencial de processamento de arquivos define como operar em dois ou mais arquivos de entrada para produzir arquivos de saída

Copyright© Apresentação 2025 por
José Augusto Baranauskas
Universidade de São Paulo

Professores são convidados a utilizarem esta
apresentação da maneira que lhes for conveniente para
uso não comercial, desde que esta nota de *copyright*
permaneça intacta.

